

Contents

UFO 5: Playtest Records and Revision Notes	1
How to read these notes	1
1. Ghost	1
2. SpinSpinSpin (Spin to Win)	3
3. Finding_Keys (Hospital Keys)	5
4. UntitledGardenGame (¡Basta Ya!)	8
5. GMTKUFO (Count Down Under)	10
Conventions used across the set	13

UFO 5: Playtest Records and Revision Notes

Michael Hardwicke GAME 1501 Rapid Prototype Development, Assignment 2 Bow Valley College, Summer 2026

How to read these notes

Five mini-games, five cycles, five different group compositions. Each cycle opened with a constraint set before the playable idea existed, so the constraint is listed first in every note. What counts as a workable mini-game changes with the constraint, so each note also states the play question the build was scoped to answer and what was deliberately left out.

The revision entries name the observed player behaviour first and the change second, because in every case where I got that order backwards the change was wrong.

1. Ghost

Constraint	One button, plus the theme “I cast”
Group	Mike (art, UI, setting), Kyle (programming)
Dates	9 June to 16 June 2026
Engine	Unity, 2D
Ship state	WebGL build on itch.io, 16 June

Scope

Play question: can a single button carry a contest between two people rather than a solo timing test?

In scope: two priests facing each other with a possessed person kneeling between them. One button each. Mash faster than your opponent to drive the spirit out and into them, because once the exorcism succeeds the spirit has to go somewhere. Score difference drives the ghost’s position and its expression.

Deliberately out of scope: movement, aiming, any second input, any AI opponent. No difficulty curve, no rounds, no score persistence. One screen, one contest.

Two constraints, not one

This cycle had a double constraint: one button, and the theme “I cast,” which was deliberately left open as to what “cast” meant. Almost the entire first day went on the second one, because “I cast” is a pun engine rather than a brief.

The ideation board is a list of things the word can mean: cast a line (fishing), cast a spell (magic), cast a limb (medical), cast out or exile (social), casting to a TV (tech), cast the ring (Lord of the Rings), casting a role and delivering a line (acting), casting out evil (exorcism). Alongside it we sketched a second axis for what a single button can physically be: timing, reflex, change, and mashing. Fishing mapped cleanly onto all four (timing to hook, mashing to reel, change to cast) and was circled first. Most of the rest were crossed out.

We landed on exorcism because it was the only reading where the pun and the button agreed on the same thing. Casting out a spirit is an act of sustained effort against resistance, which is what mashing feels like. Fishing would have been the more elegant one-button game and a weaker answer to the theme.

The decision that shaped the build was making it two player. Reading “one button” as one button *per person* rather than one button total was intentional, not a loophole we found late, and it is the reason the prototype reads without a tutorial. A ghost sliding toward whoever is losing needs no explanation. The constraint did not shrink the design space, it moved it from “what does the button do” to “who is pressing it and what happens to the loser.”

Art pipeline

I built every asset by posing figures in Maya, exporting the pose, tracing the outline, then drawing and colouring over the trace by hand. It is a crude pipeline and it looks it. It was also the only way to get consistent, correctly proportioned figures in a week without being able to draw them from imagination, and it meant the priests and the possessed person were posed in relation to each other in 3D before any of them existed as art.

Worth recording as a scoping decision rather than an art decision: the constraint on this cycle was time and my own drawing ability, and building a jig was cheaper than getting better at drawing in six days.

Revisions

Ghost reads player state, not just position. Watching people play, the ghost’s horizontal position alone did not tell either player how badly they were losing. Position is a continuous signal and players were only reading the endpoints. `GhostController.UpdateGhostSprite` now swaps between neutral, hungry and angry sprites at a score gap of 5 and 15. Discrete states read at a glance where a slide did not.

Title and button lettering redrawn. Playtesters misread letters in the hand-lettered cursive title. Commit “Update L and A for legibility,” 16 June. The pass was purely legibility, no restyling. This is the smallest revision in the set and the one with the clearest evidence: people said the wrong word out loud.

Menu state machine added after the prototype was already playable. Presses on the menus were counting toward the score. `GameManager` now gates `RegisterPress` on `GameState.Playing`, and the Start, Pause and Game Over screens route through explicit states. Commit “Add Start/Pause/Game Over Screens,” 16 June. Documented in `SCREENS_SETUP.md` in the repo root.

Class build review, 16 June. Group feedback across the cohort landed on: turn off raycast targets on UI that is not interactive, add visual indicators, add audio feedback, do not let UI

distract from play, watch audio and video timing, and stop shipping default fonts. We ran a MoSCoW pass on what remained. The raycast target note was a real bug in our build, not a style preference.

Final assessment

The prototype works. It answers its play question and it is legible to a stranger in about two seconds, which for a one button game is the whole job. It is also the weakest documented cycle of the five, and the two facts are related: I have a working build and almost no record of why any specific decision was made, so most of this note is reconstructed from commit messages and a notebook page rather than from anything written at the time.

There is a lot here we could have done better and I cannot point at most of it from evidence, only from memory. That is the finding. The prototype was fine and the practice around it was not, and it took until the fifth cycle before I was recording enough to answer this question properly.

What I would test next

Whether the ghost's expression states are doing the work or the position is, by shipping a version with the sprite swap disabled and seeing whether anyone notices.

2. SpinSpinSpin (Spin to Win)

Constraint	Theme: "Spin to Win," announced at jam start
Group	Mike, Marina, Pedro, Grace
Dates	19 June to 27 June 2026
Event	JuniperDev Game Jam, public and worldwide
Ship state	Final build, 27 June

This was the first cycle where the constraint arrived from outside the classroom. Teams were assigned before the theme was announced, so we sat as a group of four with no idea what we were building, which is a different starting condition from every other cycle in the set.

Scope

Play question: does a collection run stay tense when the play space itself is what is spinning?

In scope: a sock gremlin running in place inside a washing machine drum, like a hamster in a powered wheel. The drum turns around him and its contents stream toward him: socks in three rarity tiers to collect, and the things that actually end up in a dryer by mistake to dodge, wallets and keys and coins, plus the drum's own paddles. Timed, high score, collecting.

Deliberately out of scope: narrative, multiple levels, persistence between runs, and a gacha layer for sock rarity that we sketched and cut on day one as too much for a week. Art started as 3D with 2D assets so spawn density could be tested before anything was modelled.

Ideation and the theme filter

The brief produced: sucked into a vacuum cleaner, lazy Susan, beyblade squirrels, evil carnival rides, angry hurricane, hamster escape, sock gremlin. Marina proposed the sock gremlin and the group took it immediately, because it does three things at once. It explains the spinning (a drum), it explains the collectible (the folk joke about the machine that eats your socks), and it is funny, which on a public jam is not a small consideration.

The design consequence worth naming is that the player does not spin. The gremlin runs in place and the world rotates past him, so the theme is the treadmill rather than the character. That is what let the rest of the design be a lane runner, which is a shape we knew how to build in a week. A prototype where the *player* spins would have been a truer reading of the theme and a much worse use of seven days.

The real constraint was coordination, not the theme

Four people, all with work and family commitments, and almost no consistent in-person time. We worked remotely and we worked steadily, but the fast back and forth that a shared desk gives you was missing for most of the jam.

This did not sink the prototype and it did cap it. Everything in the revision log below that took more than one round to land took that long because the loop between “somebody noticed” and “somebody fixed it” ran through a message rather than a conversation. The single biggest change of the cycle, described below, is best understood as a response to this rather than to a gameplay problem.

Revisions

This cycle produced the cleanest evidence to change trail of the five, because feedback was delivered as a checklist and worked through as one.

Playtest round 1, 23 June. Four notes, all four shipped.

- Players were losing a run in the first second after unpausing. Added a 3 second count-down on resume.
- One obstacle contact ended the run and nobody understood why. Added a 3 hit health system with a 0.75 second grace window after each hit, so a cluster of obstacles cannot drain three hits in one frame. `RunDirector.maxHits`, `RunDirector.hitGrace`.
- Players could not tell what they had collected. The HUD now paints one stripe per sock in pickup order from `RunDirector`’s collected colour list.
- Getting hit produced no reaction at all. `HitFeedback.cs` polls `RunDirector.HitsRemaining` and fires a camera shake and a sound the moment it drops. It keys off the hit count rather than the obstacle, so every hazard added afterward inherited the feedback for free.

Playtest round 2, 29 June. Feedback recorded verbatim: start button hard to click, mode select, bigger socks and smaller items, “too fast, too easy, no roughage,” dash lines for lanes.

- **Dash lines for lanes** became `LaneStripes.cs`. The class docstring names the playtest note it came from. Scrolling dashes down each lane made the lane grid readable and, as a side effect, sold the speed, because the floor visibly rushes at you.
- **“Too fast, too easy, no roughage”** was the important one, and it was not a tuning problem. Only I could place obstacles, so difficulty could only be adjusted by whoever was holding the code, and on a team that could not get in a room together that meant every difficulty change cost a round trip of messages and a day.

The fix was SpawnWave, a ScriptableObject where a wave is painted as text: one line per row, one character per lane, o u r for sock tiers, x for obstacle, p for paddle, . for empty. Commits “Create Grid-Based Wave System” and “Update Grid Authoring System,” 25 June. After that, three people could author difficulty instead of one, asynchronously, without asking me anything.

I built this thinking it was a difficulty problem. Writing it up, it is plainly a bandwidth problem: the team’s real bottleneck was that we could not talk quickly, and a text format anyone can read and edit is a way of making a decision without a meeting. That reading also explains why it was worth spending most of a jam day on a tool during a seven day build.

- **Mode select** became WinMode on RunDirector: survive to a visible target time with socks as score, or collect a target number of socks with a hidden timer that tightens spawn spacing as you go. Two playstyles, switchable in the Inspector, so we could put both in front of players rather than argue about which was better.

Playtest round 3. Speed up at the end, paddles across the whole screen, plain socks with fewer rare duplicates, game over screens. All shipped: RunDirector acceleration to a max speed, “4 Full Length Paddles, always spawn in Center Lane,” “Add Plain Sock,” “Game Over Screens.” Two notes were rejected as already handled: the camera being was already pushed, and the logo tweak already existed.

Accessibility, unprompted by playtest. AccessibilityManager persists a high contrast preference through PlayerPrefs and broadcasts it as an event so no visual system needs a direct reference, plus a ColorAccessibilityChecker editor tool. This was a team convention rather than player evidence, and it is the one change in this cycle I cannot trace to an observation.

3. Finding_Keys (Hospital Keys)

Constraint	A despised game mechanic, drawn at random from scraps of paper in a hat
Drawn	“Finding keys for lock puzzles, and use key”
Brief	Take the despised mechanic and make it enjoyable
Group	Mike (level design, environment, build), Danish (systems, AI, UI)
Dates	7 July to 14 July 2026
Ship state	WebGL build, 14 July

This is the only cycle where the constraint was adversarial by design. Other groups drew other mechanics. Ours was the key hunt, and the assignment was not to work around it but to make somebody enjoy it.

Scope

Play question: the key hunt is boring because collecting a key is not a decision. Can it become one if the keys are the means to a goal that is not “open the last door”?

In scope: a top down hospital. Colour coded keycards found by searching containers, colour coded doors that open floor sections, an escaped alien that flees the player through

hatches, and two cardboard boxes that can be slid over hatches. Herd the alien into a section, seal both its hatches, and it is trapped. That is the win.

Deliberately out of scope: combat, inventory beyond the card tier, more than one floor, any fail state for the player.

Ideation

The 7 July board ran through: keys that make bigger keys, locking doors to stop a chasing monster, a security guard with a keyring unlocking doors for a VIP, a valet hunting a client's key, cryptography and Wordle style decryption, matching a musical key to play alongside a musician, finding words to open multiple locks, and escaping a castle. Super Hexagon came up as the reference for how a despised structure can be redeemed by pacing.

We took a version of “lock doors to stop a chasing monster” and inverted it. The alien flees rather than chases, so the keys are not a defence, they are how you cut off its options. That inversion is the whole attempt at the brief: a key you collect is boring, but a key that closes one of the alien's exits is a move in a hunt. Whether it lands is a separate question, addressed below.

We took the chase framing. The hospital gave us a natural severity ladder for access, planned on paper as six tiers: Green 1, Blue 2, Yellow 3, Red 4, Silver 5, Gold 6.

The ladder was cut to five during the cycle. The shipped `KeycardLevel` enum is Blue, Red, Purple, Silver, Gold, with Green and Yellow dropped and Purple added. Commit “sprite slots for 5 key cards,” 14 July. Six colours was more than a one week prototype could carry: each tier needs its own card art, its own door, and a place in the hospital where it makes sense to find it, and six of those is content volume rather than design. `PlayerInventory.UpgradeKeycard` only ever moves the player up the ladder, so the tier count is also the run length, and cutting a tier shortened the hunt as a side effect.

The revision that defined the cycle

The alien could get cornered by accident, and the win condition was rewritten so it could not.

Commit “Removed cornered state, finalised code,” Danish, 13 July, the evening after a live playtest.

Before that commit, `FindAndFleeToNearestHatch` ended with a fallback: if the alien could not find a hatch it could reach, it triggered the Cornered state. That is cornering as a pathing failure. The alien would wedge itself somewhere, fail its search, and declare itself trapped, and the player won without having done anything. In a prototype whose entire subject is the key hunt, the fastest route to a win did not involve keys at all. It involved the alien making a mistake.

The commit rewrote it so cornering is only ever player-authored. The relevant line is commented in the source as a “fail-safe cheat”: the alien is only allowed to be cornered when `CardboardBox.BlockedHatchCount >= 2`, checked at the top of both flee routines. Three supporting changes make the accidental route impossible:

- A back-up search was added. If the directional filter rejects every hatch, the alien now takes any active hatch at all rather than giving up. It always has somewhere to go.
- The dot product check that stopped the alien fleeing toward the player was relaxed to `dot < 0.0f && hits.Length > 1`, so with only one hatch available it will run past you rather than stall.
- A break-free was added to the Cornered case itself. Pull a box away and the alien is loose again on the same frame.

The net effect is that the two boxes are now the only thing that can end the run, and reaching both boxes requires the coloured cards. The key hunt was made load bearing by removing every other way to win.

What I had wrong until I checked the diff. Writing the first draft of this note from the commit message alone, I recorded this as the box trapping being *removed* because playtesters were abusing it. That is the opposite of what happened, and it took ten minutes with `git show` to find out. The commit message “Removed cornered state” describes the accidental cornered state, not the designed one, and I read it as the designed one because that was the more interesting story. Worth recording as a research failure alongside the design ones.

Other playtest driven changes

- “Playtested, fixed bugs, updated AlienAI script,” 13 July. Danish’s commit after a live session.
- “Move player and alien further apart,” 14 July. The alien’s `playerDetectRadius` of 5 units meant it spotted the player and fled on the first frame, so nobody ever saw the idle state. The fix was spawn placement, not a code change.
- “Reduced collider on player to move through doors,” 9 July. Players were getting stuck in doorways. A collider size problem reading as a level design problem.
- Search feedback: `SearchableItem` announces an already searched container by name through `GameUIManager.DisplayNotification` rather than silently doing nothing, and plays a search sound the frame the interaction lands. Players were re-searching the same cabinet because the game gave them no reason to remember it was empty.

Instructor feedback, 15 July. No default Unity font, “doesn’t look like a hospital,” “almost works but doesn’t is unsatisfying,” and “brute-forcing is a big no-no.”

Two things about this entry are worth stating plainly.

First, the final commit on this repo is 14 July. This feedback arrived after the cycle had closed and drove no revision here. It is recorded because “brute-forcing is a big no-no” is independent confirmation of the change we had already made on 13 July for our own reasons. Accidental cornering was a brute force win, we closed it on playtest evidence, and the same principle came back from the room two days later without anyone connecting the two.

Second, “almost works but doesn’t is unsatisfying” is the sharpest note anyone gave me all term. It names the failure mode a key hunt invites better than I could have. A door that almost opens is not a puzzle, it is a taunt, and a search that turns up nothing is the same taunt in slower motion. Our answer to the brief was to give the keys a purpose beyond the last door, which is a real answer, and it does not touch this problem at all. If the prototype had another cycle, this is what I would work on.

The same session set the standard for the following build: a game loop, mechanics with explicit objectives and win and lose states, and robust state machines. That went straight into how I scoped the next two prototypes.

Final assessment

Simple premise, and the execution could have been much better. The alien is legible for about thirty seconds and then becomes a thing that runs away for reasons the player cannot read, because nothing communicates why it chose the hatch it chose. The hospital does not look like a hospital. The searching gives no information: an empty cabinet and a cabinet you have not opened yet look identical from across the room, so the hunt is exhaustive rather than deductive.

That last one is the actual reason key hunting is despised, and I did not see it during the build. We treated the problem as pacing, so we added a chase. The deeper problem is that a key hunt gives the player no way to rule anything out. Search is not a decision if every container is equally likely. Of the five cycles this is the one I most want to run again, because the brief was the most interesting and my answer to it was the most partial.

4. UntitledGardenGame (¡Basta Ya!)

Constraints	Two. No original art, steal everything with attribution. And the pair had to agree on something they both genuinely disliked, and build about that
Prompt	The “mad as hell” scene from <i>Network</i> (1976). A newscaster gets up from the desk and tells the audience to stop accepting things as they are
Pair	Mike (programming, systems), Du (art direction, UI, game design). Turbo Speelo credited for writing and lore
Dates	14 July to 21 July 2026
Event	UFO IV: Mad as Hell jam
Ship state	Windows build, 21 July

Scope

Play question: can the slow, unglamorous work of organising feel like a game, when the fun parts of revolution (the rally, the march) are the parts that cost the most and produce the least on their own?

In scope: a revolution management sim, part idle game and part strategy game. Build a grassroots resistance day by day: recruit from your community, feed them, train them, write and distribute information, hold rallies, protests and marches. Queue actions against a limited pool of daily hours, watch them tick down, spend resources and receive different ones back.

Deliberately out of scope: combat, individually named characters, anything real time outside the queue tick.

The constraint nobody mentions: agreeing on the anger

The art rule is the one that sounds like the constraint, and the harder one was that both of us had to land on a shared object of dislike before anything could be designed. A jam prompt that says “be angry” does not work on a pair until the pair is angry about the same thing.

Our ideation ran through wealth disparity, resource overuse, FOMO, dragon sickness, forced choices, speculative bubbles from the 1920s through dot com and housing, conspiracy theories, machine learning and algorithmic curation, with Dan Olson’s *Line Goes Up* as the shared reference point. What we converged on was less a topic than a shape: systems that extract from people who have no way to opt out. That is why the game is about mutual aid infrastructure rather than about street fighting, and why the win condition is a functioning commune rather than a toppled government.

This constraint did more to determine the design than the art rule did, and I would not have said so at the time.

How the art constraint changed the pipeline

“Steal everything” sounds like a shortcut and is not. It moves the art job from drawing to sourcing and composing, and it makes bookkeeping a shipping requirement rather than a courtesy. CREDITS.md in the repo root carries library level sources, per asset attribution links live on the itch page, fonts are listed with licences, and there is an explicit AI disclosure covering code, upscaling and writing.

It also pushed the aesthetic somewhere we would not have gone otherwise. Collage from stock photography and vectors landed on a 1970s broadcast look, which matched the Network prompt exactly. The constraint chose the visual language and the visual language chose the tone.

The economy

Worth setting out, because the revisions below only make sense against it. Four variables: Community, Machine, Food, People. Three action types, each with one job:

- **Community** refills the Community bar. Care work. Tending the garden, producing medicine, mutual aid.
- **Organize** adds People. Recruiting, zines, debates, festivals.
- **Resist** drains the Machine bar. Tearing down propaganda, strikes, protests.

The loop that makes it a game is that daily action points are $\text{base} + \text{People} / \text{peoplePerBonusPoint}$. Growth literally buys time. Recruiting is not a score, it is the only way to get more done tomorrow, so the player is constantly choosing between acting now and being able to act more later. Actions sit on a four tier tech tree with prerequisites and supporter gates at 15, 40 and 100.

Balancing the time cost, consumption and production of about twenty actions across four tiers was the bulk of the design work and the part I am happiest with.

Revisions

Hybrid time: the clock only runs when the belt is loaded. The design pulled in two directions. An idle game wants a clock that always runs, so time pressure is real. A strategy game wants the player to plan without being punished for thinking. Early builds ran the clock continuously and playtesters either rushed their planning or resented the pressure.

The resolution is that the day clock is frozen while the action queue is empty and runs at SecondsPerHour only while there is something on it. Plan in peace, and the moment you commit, time starts moving. It behaves like a factory belt that only advances when something is on it.

This is the single decision that made the two genres coexist, and it came from watching people hesitate rather than from anyone arguing about genre.

Costs settle on completion, not on queueing. Charging up front meant cancelling a queued action needed a refund path, which was fiddly and produced rounding bugs. Moving the settle to completion removed the refund entirely. The tradeoff is that a resource can drain out from under a queued action, so an action can become unaffordable while waiting; it is skipped and a journal note explains why. Choosing which failure mode to keep, rather than trying to have neither, was the right call for a week.

Days felt like days, not years. The design goal written on the ideation page was “make the days feel like years.” In play they did not, because a day took real minutes to grind through. Commit “Make ingame time faster,” 21 July. The fix was pacing, not content.

Resource changes were invisible. Players were queuing actions and not noticing what those actions cost them. Commits “Resource Loss/Gain Indication” and “Reorder Resources,” 21 July. Same failure as the sock HUD on the previous cycle: the system was working and the player could not see it working.

Content authoring moved out of code. Three editor windows were built during the jam: RevTechTreeWindow with a prerequisites editor, RevEndingMapWindow for visualising which resource states reach which ending, and RevContentGenerator. Du could then write actions, news events and endings as ScriptableObjects without waiting on me. This is the same move as SpawnWave two cycles earlier, and on a two person team it was the difference between one content author and two.

Font selection in options. OpenDyslexic shipped as a selectable option, commits “Add font changing capability in options” and “Font Update,” 21 July. Partly a response to Sean’s standing complaint about default Unity fonts, partly accessibility.

Late file splitting. Commits “Refactor to reduce file lengths,” 20 July, and “Make files less than 300 lines,” 21 July. RevGameManager became partial classes across Days, News and Queue; RevGameScreenController split seven ways. This was mechanical splitting of a working system, not a rewrite, and it cost almost nothing. Note this for the next cycle, where the distinction mattered a great deal.

Final assessment

This is the prototype I would most like to continue, and the only one of the five where I think the core is actually finished rather than merely working. The economy holds up: twenty odd actions across four tiers, balanced for time cost against consumption and production, with growth buying time so the player is always trading present output against future capacity. That took most of the week and it does what it should.

What is missing is everything above the economy. The tech tree ends rather than branching meaningfully, the news events land on the player rather than responding to what they built, and the endings are reached rather than earned. A player who understands the loop has nothing left to discover on a second run. That is the work I would pick up if I came back to it, and I intend to.

5. GMTKUFO (Count Down Under)

Constraint	Theme: Countdown. Four days, not seven
Group	Mike (systems), Yuki (programming), Marina (art), Kyle (modelling)
Dates	22 July to 26 July 2026
Event	GMTK Jam 2026, global, alongside teams outside the class
Ship state	Build #5, 26 July
Status	Continuing as the Assignment 3 project

The outlier in the set. Every other cycle was a week. This one was four days, and that difference did more damage than the theme or the team composition did. It is also the only prototype we are carrying forward rather than closing.

Scope

Play question: does a countdown still create pressure if the player chooses when to stop?

In scope: a side scrolling level. You are Count Dracula, in Australia, as a tourist, because “Count Down Under” is a pun and we are not sorry. Move through a night city draining blood from victims, avoid hazards both stationary and moving, kangaroos and crocodiles among them, and reach the coffin at the end of the level before the sun finishes you. Blood is score and blood is health, so the run is a wager: gather more and arrive slower, or sprint for the end with nothing in the tank.

Deliberately out of scope: multiple levels, progression between runs, any story.

The theme reading, and where the design doc and the build diverged

The obvious reading of “countdown” is a fail timer. We wanted it as a decision timer. The GDD written on day one describes a run with no fixed ending, where the player sleeps in a coffin *whenever they choose* and banks whatever they still hold, and daylight is a drain rather than a death.

What got built is a level with a finish line. Coffin.cs carries an `_endsRun` flag whose tooltip distinguishes “the coffin at the end of the level” from “the one the player starts in,” which is a race, not an open-ended bender. The two readings are not the same game. The open one is about greed and knowing when to quit. The built one is about routing a level under a clock.

Neither is wrong and we never chose between them out loud. The design doc kept describing one game while the level was quietly built as the other, and nobody noticed because for most of four days there was no playable level to compare the document against. Recording this as the most useful finding of the cycle: a GDD stops being true the moment nobody is checking it against a build, and it does not announce when that happens.

Revisions

This cycle is the best documented of the five, because I kept a running debug log (DEBUG_NOTES.md) with dates and measurements rather than relying on memory.

After playtest 1: the bite would not hold. Players walked out of a bite and the victim died anyway. `PSEating.Execute` only checked whether the drain had finished, so entering the state committed to the full drain regardless of where the player ended up. The drift itself was caused by Enter snapping the player to the victim’s exact x while they already shared a z, so two colliders occupied the same space and physics shoved them apart.

Two changes: the snap was removed, since the overlap query had already proven the victim was in reach, and the player is now locked horizontally for the duration of the drain. A bite became a two second commitment, measured at 167 frames for a 100 blood victim at drain speed 50.

That fix started as a bug and ended as a design decision. Being pinned for two seconds next to a kangaroo is a real risk, which gave the hazards a job they did not have before.

After playtest 2: the bite animation played at a distance. `_boxCastHalf.x` is 1.0, so the bite triggered from a full unit away and the attack animation played across a visible gap. `PSEating.Enter` now pulls the player in to a `BiteStandoff` of 0.75 on whichever side they approached from. The floor is roughly 0.7, being the two capsule radii summed, and going below it reproduces the original interpenetration shove.

The instrument lied, twice. Worth recording separately because both cost real time.

BiteDebug reported a drained victim as “in range on all axes, so the miss is the layer mask,” because `OverlapBox` skips disabled colliders and `Die()` disables the victim’s. The tool confidently pointed at the wrong subsystem.

Separately, a full victim read as a gain of 99 rather than 100. I read that as a drain bug. It was `Mathf.FloorToInt` on a float score that summed to 199.99998. Changed to `RoundToInt`.

Blood and health merged into one pool. Two resources that always moved together. `MaxHealth`, `_currentHealth`, `_healSpeed` and `HealthNormalized` came off `PlayerController`; damage now spends blood through `GameManager.RemoveBlood`, which ends the run at zero. The design consequence was that the daylight drain can now kill you, which it could not before. Bleeding out in the sun ending the run is consistent in a way the two pool version was not.

A dead end, recorded so it is not repeated. Replacing the animator controller swapping with a single parameter driven controller was attempted on 25 July and never worked. It cost most of an afternoon in a four day jam. The refactor is still the better design, but it needs a branch and testing time, not a jam afternoon.

The trap that made it worse: `Drac.controller` had been renamed from `Idle.controller` and kept its guid, so the Animator window showed the correct controller bound even while the old swapping code was still the thing running. The confirmation was false and I acted on it.

Set against the file splitting on the previous cycle, the line is clear enough to write down as a rule. Splitting a working system into smaller files is safe under time pressure. Changing how a working system works is not.

Final assessment

The game was not close to functional and the level design suffered badly. That is the summary, and everything above should be read against it: the revision log is detailed because the build was broken, not because the process was good. `DEBUG_NOTES.md` is the longest document I produced all term and it is a record of firefighting, not of design.

The obvious explanation is the four days, and I do not think it is the right one.

Look at what we committed to on day one: a 3D character controller with a full state machine, two player forms with an animator swap between them, three hazard types, a dynamic sun driving real shadows, shadow detection on the player, victims with drain states, and a HUD tracking a resource that is simultaneously score and health. The GDD lists an MVP and then five stretch goals underneath it. We did not hit the MVP.

Seven days would not have fixed that, it would have produced a slightly less broken version of the same overreach. The scope was set for a week on day one and the calendar said four days, and nobody adjusted the plan when the number changed. The MVP list was written and then not used as a cutting tool, which is the only thing an MVP list is for.

The bite fixes, the merged resource pool and the recorded dead end are all real and all of them are repairs. Not one of them is a change driven by watching somebody play, because we barely got somebody playing. That is the difference between this cycle and *SpinSpin*, and it is not a difference in hours available.

Carried forward

This becomes the Assignment 3 project. What survives: the pun, the wager between gathering and running, blood as a single pool, and the bite as a committed two second action. What needs deciding before anything else is built is the question the GDD and the level answered differently, whether this is an open ended bender or a race to a finish line, because level design cannot be fixed until that is settled.

Conventions used across the set

- _Project root folder, with Code split into Core, Gameplay and UI, on every build from Ghost onward.
- Explicit state machines rather than boolean flags, after the 15 July session named robust state machines as a requirement.
- Buckets and ScriptableObjects for anything a non-programmer needs to author.
- Playtest notes recorded as observed behaviour first, change second.
- Asset attribution tracked in repo, not only on the submission page.